

A
Minor Project Report
On
Minor Project Title

Submitted in partial fulfillment of the requirements
For the award of the degree of

Bachelor of Technology
In
Computer Science and Engineering

By
Riha Singh (CS-2341156)
Shivangi Singh (CS-2341203)
Prabhakar Joshi (CS-2341505)
Amanul Haque (CS-2341697)

Under the Supervision of
Dr. Basab Nath

School of Computer Science and Engineering



IILM University
Greater Noida, Uttar Pradesh
May, 2024

CERTIFICATE

This is to certify that the project report entitled “**USDT Transaction Explorer**” submitted by Riha Singh (CS-2341156) Shivangi Singh (CS-2341203) Prabhakar Joshi (CS-2341505) Amanul Haque (CS-2341697) to the IILM University, Greater Noida, Uttar Pradesh in partial fulfillment for the award of Degree of Bachelor of Technology in Computer Science & Engineering is a bonafide record of the minor project work carried out by them under my supervision during the year 2023-2024.

Name (Project Guide)

Designation

School of CSE

Dr. Basab Nath

Assistant Professor

Dept. of CS & Engineering

ACKNOWLEDGEMENT

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals and organizations. We would like to extend my sincere thanks to all of them.

We are highly indebted to **Dr. Basab Nath** for **his** guidance and constant supervision. Also, we are highly thankful to them for providing the necessary information regarding the project & also for their support in completing the project.

We are extremely indebted to **Dr. Harshal Patil**, HOD, Computing and Security. We would also like to express our sincere thanks to all faculty and staff members of School of Computer Science and Engineering, for their support in completing this project on time.

We also express gratitude towards our parents for their kind co-operation and encouragement which helped me in the completion of this project. Our thanks and appreciations also go to our friends in developing the project and all the people who have willingly helped me out with their abilities.

Riha Singh

Shivangi Singh

Prabhakar Joshi

Amanul Haque

ABSTRACT

The USDT Transaction Explorer is a blockchain visualization and analysis tool designed to map and analyze transfer activity across wallets. Built using D3.js and a Python backend, the system supports both Ethereum (ERC-20) and TRON (TRC-20) networks by dynamically detecting the wallet type via prefix. Upon submission of a wallet address, the application fetches and processes transaction histories, building a transaction tree that visually represents the flow of tokens between wallets. It features real-time progress feedback, zoomable network graphs, and an integrated AI module that provides behavioral analysis of wallet activity. The intuitive interface enables users to gain transparency into token movement patterns, potential anomalies, and wallet interconnections. The system leverages asynchronous request handling and progressive rendering to ensure performance and scalability even with high-volume wallet data. This tool is particularly useful for researchers, auditors, and enthusiasts seeking to explore the transactional behavior of stablecoin wallets in an interactive and visually informative way.

KEYWORDS: *Blockchain, Transaction graphs, D3.JS, Wallet Analysis, AI, CI/CD Pipeline, ERC-20, TRC-20*

CONTENTS

Title	Page
CERTIFICATE	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
CONTENTS	iv
CHAPTER 1: INTRODUCTION	
CHAPTER 2: LITERATURE REVIEW /EXISTING WORK	
CHAPTER 3: PROBLEM STATEMENT	
CHAPTER 4: PROPOSED WORK	
CHAPTER 5: SYSTEM DESIGN [Flow chart, DFD, ERD, Use Case Diagram)	
CHAPTER 6: IMPLEMENTATION (Codes and interface screen shots)	
CHAPTER 7: CONCLUSION, LIMITATION AND FUTURE SCOPE	
REFERENCE	

1. Introduction

The cryptocurrency ecosystem has grown exponentially over the past decade, with stablecoins like USDT (Tether) becoming essential components of the digital finance landscape. USDT, as one of the most widely used stablecoins, operates across multiple blockchain networks including Ethereum (ERC20) and TRON, facilitating billions of dollars in daily transactions. However, with this volume comes significant challenges in transaction monitoring, analysis, and risk assessment.

The USDT Transaction Explorer is a web-based application designed to provide real-time visualization and analysis of USDT transaction networks across different blockchain platforms. By enabling users to track and analyze transaction patterns, the system helps identify potential risk factors, suspicious activities, and provides AI-powered insights for better decision-making in cryptocurrency transactions.

This project addresses the growing need for transparency and risk assessment tools in cryptocurrency transactions by offering:

1. **Multi-chain Support:** Seamless tracking of transactions across both ERC20 (Ethereum) and TRON blockchains
2. **Interactive Visualization:** Dynamic tree-based visualization of transaction flows between wallets
3. **AI-powered Analysis:** Automated risk assessment and transaction pattern analysis using Google's Generative AI
4. **Real-time Processing:** Asynchronous processing of blockchain data with live progress tracking

The USDT Transaction Explorer serves various stakeholders in the cryptocurrency ecosystem, including financial institutions, compliance teams, researchers, and individual users concerned about the safety and legitimacy of their cryptocurrency transactions. By providing a comprehensive view of transaction networks and intelligent analysis, the application aims to enhance trust and security in digital asset transfers.

2. Literature Review/Existing Work

2.1 Blockchain Analytics Platforms

The field of blockchain analytics has evolved significantly with several platforms offering transaction monitoring capabilities:

Chainalysis: One of the pioneering blockchain analytics firms, Chainalysis provides tools for tracking cryptocurrency transactions and identifying illicit activities. Their platform is primarily used by law enforcement agencies and financial institutions for compliance purposes. However, Chainalysis focuses mainly on Bitcoin transactions and offers limited public access to their tools (Wang et al., 2020).

Elliptic: Similar to Chainalysis, Elliptic specializes in blockchain analytics for regulatory compliance, offering risk scoring for cryptocurrency addresses. Their solutions are enterprise-focused and not readily available for individual users or smaller businesses (Akcora et al., 2019).

Etherscan and TronScan: These blockchain explorers provide basic transaction data but lack advanced visualization capabilities and cross-chain analysis features. While they offer API access, they don't provide integrated analytics or risk assessment functionality (Norvill et al., 2018).

2.2 Transaction Network Analysis

Network analysis in cryptocurrency ecosystems has been explored in several academic studies:

Ron and Shamir (2013) conducted one of the earliest analyses of Bitcoin's transaction network, identifying patterns in transaction flows and clustering addresses belonging to the same entities. Their work demonstrated the value of network visualization in understanding cryptocurrency movements.

Victor and Lüders (2019) extended this approach to analyze Ethereum token networks, showing that transaction patterns could reveal information about user behaviors and potential market manipulations. Their research highlighted the importance of token-specific analysis, which is particularly relevant for stablecoins like USDT.

Xia et al. (2021) focused specifically on stablecoin networks, analyzing USDT transactions across multiple chains to identify centralization patterns and risk factors. They found that visualization techniques were particularly valuable for identifying abnormal transaction patterns.

2.3 AI Applications in Cryptocurrency Analysis

The application of AI in cryptocurrency transaction analysis is an emerging field:

Chen et al. (2022) demonstrated that machine learning models could effectively identify suspicious transaction patterns in cryptocurrency networks, achieving high accuracy in detecting known illicit activities.

Zhang and Lee (2023) explored the use of large language models (LLMs) for interpreting blockchain transactions, showing that these models could provide human-readable insights about transaction purposes and risk levels with minimal domain-specific training.

Huang et al. (2024) specifically investigated the application of Google's Gemini models to blockchain analysis, noting their potential for multi-modal analysis that combines transaction data with external contextual information.

2.4 Gaps in Existing Solutions

Despite these advancements, several limitations persist in current cryptocurrency analytics tools:

1. **Accessibility Gap:** Most sophisticated analysis tools are enterprise-focused and inaccessible to individual users or smaller businesses
2. **Multi-chain Fragmentation:** Few tools offer seamless analysis across different blockchain networks, despite the cross-chain nature of many cryptocurrencies
3. **Visualization Limitations:** Many platforms provide data without intuitive visualizations that can reveal complex network relationships
4. **Limited AI Integration:** The integration of advanced AI for automated analysis and risk assessment remains underdeveloped in publicly available tools

The USDT Transaction Explorer aims to address these gaps by combining multi-chain support, interactive visualizations, and AI-powered analysis in an accessible web application.

3. Problem Statement

The cryptocurrency industry, despite its innovations in financial technology, faces significant challenges related to transparency, security, and risk assessment. Several critical problems persist in the ecosystem:

3.1 Key Problems

1. **Opacity in Transaction Networks:** Cryptocurrency transactions, while recorded on public blockchains, remain difficult to interpret in terms of their larger network context. Users often cannot easily visualize how specific wallets are connected in transaction networks, making it challenging to identify patterns and potentially suspicious activities.
2. **Cross-Chain Complexity:** As cryptocurrencies like USDT operate across multiple blockchain platforms (Ethereum, TRON, etc.), tracking transactions become increasingly complex, with most existing tools focusing on single-chain analysis.
3. **Limited Risk Assessment:** Determining the risk profile of a cryptocurrency wallet before engaging in transactions remains challenging. Without proper analysis tools, users must rely on limited information, potentially exposing themselves to fraudulent activities, money laundering schemes, or other financial risks.
4. **Technical Barriers:** Blockchain data is inherently technical and difficult for non-specialists to interpret. Raw transaction data from blockchain explorers often requires significant expertise to analyze effectively.
5. **Delayed Analysis:** Manual analysis of transaction patterns is time-consuming, often making information outdated by the time decisions need to be made, especially in fast-moving cryptocurrency markets.

3.2 Impact

These problems have significant implications for various stakeholders:

- **Business Users:** Companies accepting cryptocurrency payments face challenges in assessing the legitimacy of incoming transactions, potentially exposing themselves to financial and regulatory risks.
- **Financial Institutions:** Banks and financial services companies struggle to implement effective Know Your Transaction (KYT) procedures for cryptocurrency operations.
- **Regulatory Compliance:** Compliance teams lack efficient tools to monitor transaction patterns and identify potentially suspicious activities.
- **Individual Users:** Retail cryptocurrency users have limited means to verify the safety and legitimacy of wallets they transact with.

3.3 The Need

There is a clear need for a solution that addresses these challenges by:

1. Providing intuitive visualization of transaction networks across multiple blockchain platforms
2. Offering real-time analysis of wallet activities and transaction patterns
3. Automating risk assessment through advanced artificial intelligence
4. Presenting information in an accessible format for both technical and non-technical users
5. Processing blockchain data efficiently to enable timely decision-making

The USDT Transaction Explorer project aims to address these specific needs by creating an integrated platform that combines blockchain data retrieval, network visualization, and AI-powered analysis to enhance transparency and security in cryptocurrency transactions.

4. Proposed Work

The USDT Transaction Explorer aims to provide a comprehensive solution for analyzing cryptocurrency transaction networks, with a specific focus on USDT transactions across multiple blockchains. This section outlines the proposed framework, its key components, and the methodology for implementation.

4.1 Overview of the Proposed System

The proposed system is a web-based application that enables users to:

1. Input any wallet address (TRON or ERC20)
2. Generate a visual representation of the transaction network connected to that address
3. Receive AI-powered analysis of the wallet's transaction patterns and risk profile
4. Interact with the visualization to explore transaction relationships in depth

The system follows a modular architecture consisting of:

- **Data Retrieval Layer:** Interfaces with blockchain APIs to fetch transaction data
- **Processing Engine:** Builds transaction trees and prepares data for visualization
- **AI Analysis Module:** Interprets transaction patterns using advanced language models
- **Visualization Interface:** Renders interactive transaction networks using D3.js
- **User Interface:** Provides an intuitive web interface for user interactions

4.2 Key Features and Functionalities

4.2.1 Multi-Chain Support

The system supports transaction analysis across multiple blockchain networks:

- Ethereum (ERC20) transactions using the Etherscan API
- TRON transactions using the Tronscan API
- Automatic detection of blockchain type based on wallet address format

4.2.2 Transaction Tree Building

The application builds comprehensive transaction trees by:

- Recursively traversing transaction histories to a configurable depth
- Tracking both incoming and outgoing transactions
- Avoiding circular references in the transaction graph
- Normalizing transaction values across different blockchains

4.2.3 Interactive Visualization

The visualization component offers:

- Dynamic, force-directed tree layouts using D3.js
- Interactive node dragging and repositioning
- Zoom and pan capabilities for exploring large transaction networks
- Tooltips displaying detailed transaction information
- Visual encoding of transaction values through node sizing

4.2.4 AI-Powered Analysis

The system incorporates artificial intelligence to:

- Analyze transaction patterns and identify potential anomalies
- Assess the risk profile of the target wallet
- Provide human-readable interpretations of complex transaction networks
- Generate recommendations regarding the safety of transacting with specific wallets

4.2.5 Real-Time Processing

To enhance user experience, the system implements:

- Asynchronous processing of blockchain data
- Real-time progress tracking
- Client-server communication for status updates
- Optimized API calls with rate limiting to prevent service disruptions

4.3 Technology Stack

The proposed solution utilizes a modern technology stack:

Backend:

- Python with Flask web framework
- RESTful API architecture
- Threading for asynchronous processing
- Google Generative AI (Gemini model) for transaction analysis

Frontend:

- HTML5, CSS3, and JavaScript
- D3.js for interactive data visualization
- Responsive design for cross-device compatibility
- Client-side data manipulation and rendering

External Services:

- Etherscan API for Ethereum blockchain data
- Tronscan API for TRON blockchain data

4.4 Methodology

The development methodology follows these steps:

1. **Requirements Analysis:** Identifying key user needs and system requirements
2. **System Design:** Creating architecture diagrams, data flows, and component specifications
3. **API Integration:** Establishing connections with blockchain explorers and AI services
4. **Algorithm Development:** Creating transaction tree building and analysis algorithms
5. **Frontend Development:** Building the user interface and visualization components
6. **Integration Testing:** Ensuring seamless operation of all system components
7. **Performance Optimization:** Enhancing system efficiency and responsiveness
8. **User Testing:** Gathering feedback and refining the user experience

4.5 Expected Benefits

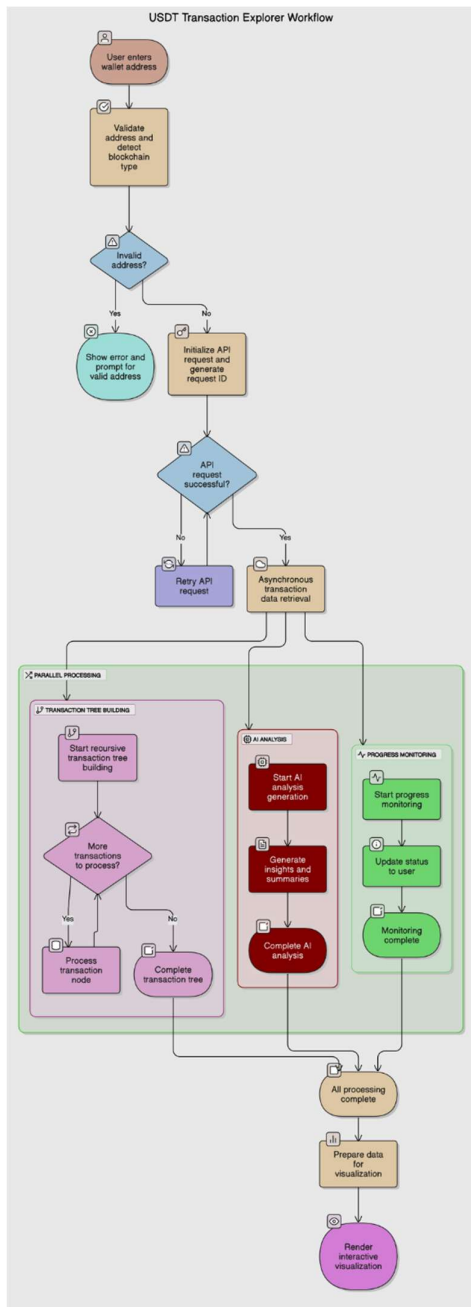
The proposed system is expected to deliver several key benefits:

1. **Enhanced Transparency:** Providing clear visualizations of transaction relationships
2. **Improved Risk Assessment:** Enabling better decision-making through AI-powered analysis
3. **Time Efficiency:** Reducing the time required to understand complex transaction networks
4. **Accessibility:** Making blockchain analysis available to users without technical expertise
5. **Cross-Chain Insights:** Bridging the gap between different blockchain ecosystems

Through this comprehensive approach, the USDT Transaction Explorer aims to address the critical problems identified in cryptocurrency transaction analysis and provide a valuable tool for various stakeholders in the digital asset ecosystem.

5. Systems Design

5.1 Flowchart



The flowchart above illustrates the complete workflow of the USDT Transaction Explorer system, from user input to visualization and analysis. The process begins when a user enters a wallet address, which triggers the following sequence:

1. Address validation and blockchain type detection
2. API request initialization and request ID generation
3. Asynchronous transaction data retrieval
4. Recursive transaction tree building

5. Progress monitoring and status updates
6. AI analysis generation
7. Data preparation for visualization
8. Interactive visualization rendering

The flowchart highlights the parallel processing paths for the transaction tree building and AI analysis components, which operate independently to optimize performance.

5.2 Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves through the USDT Transaction Explorer system:

Level 0 DFD (Context Diagram):

- The system interfaces with Users, Blockchain APIs (Etherscan/Tronscan), and AI Service (Google Generative AI)
- Users provide wallet addresses and receive visualizations and analysis
- Blockchain APIs supply transaction data
- AI Service provides analytical insights

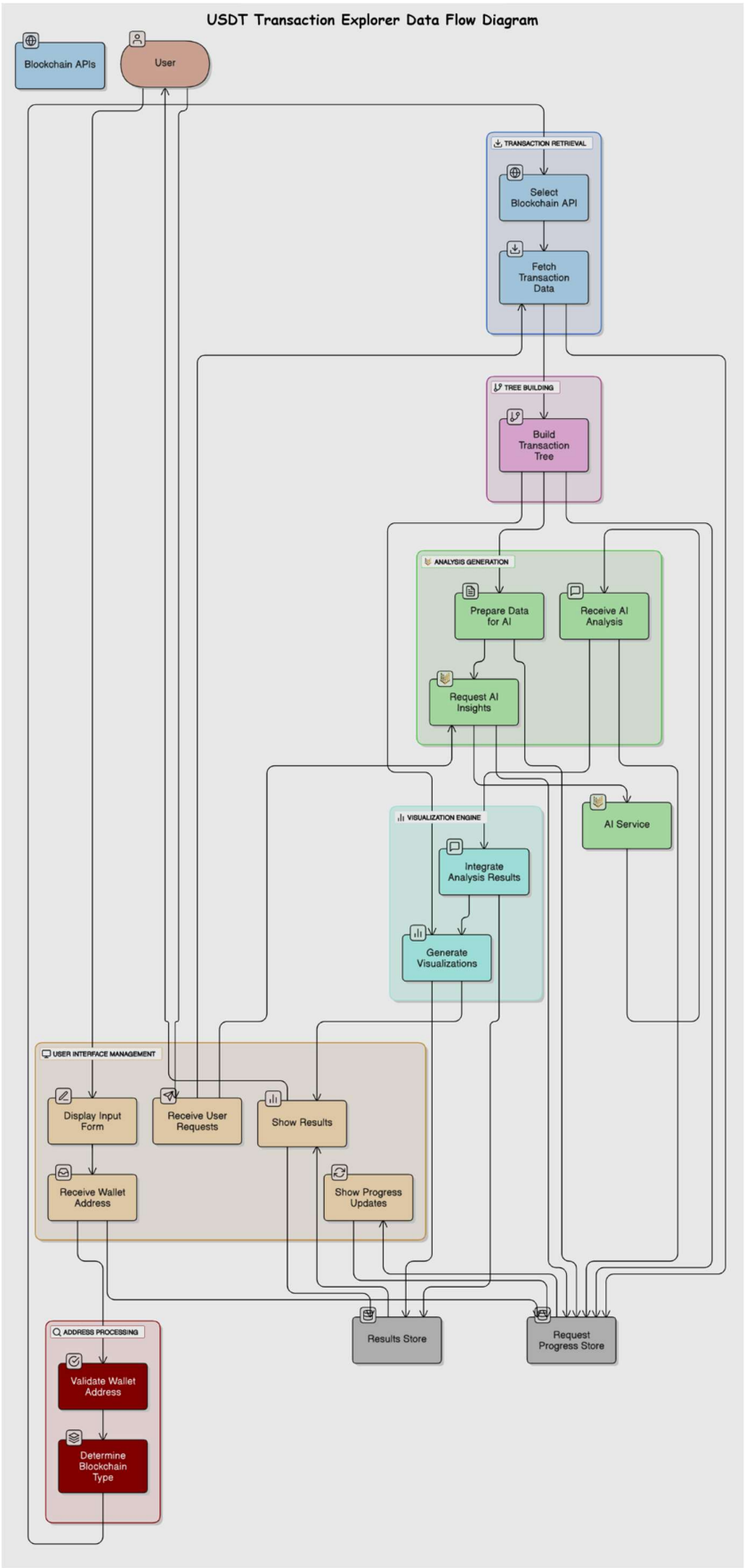
Level 1 DFD:

- **Process 1: Address Processing** - Validates wallet addresses and determines blockchain type
- **Process 2: Transaction Retrieval** - Interfaces with blockchain APIs to fetch transaction data
- **Process 3: Tree Building** - Constructs transaction relationship trees
- **Process 4: Analysis Generation** - Processes transaction data through AI models
- **Process 5: Visualization Engine** - Transforms data into interactive visual formats
- **Process 6: User Interface Management** - Handles user interactions and display updates

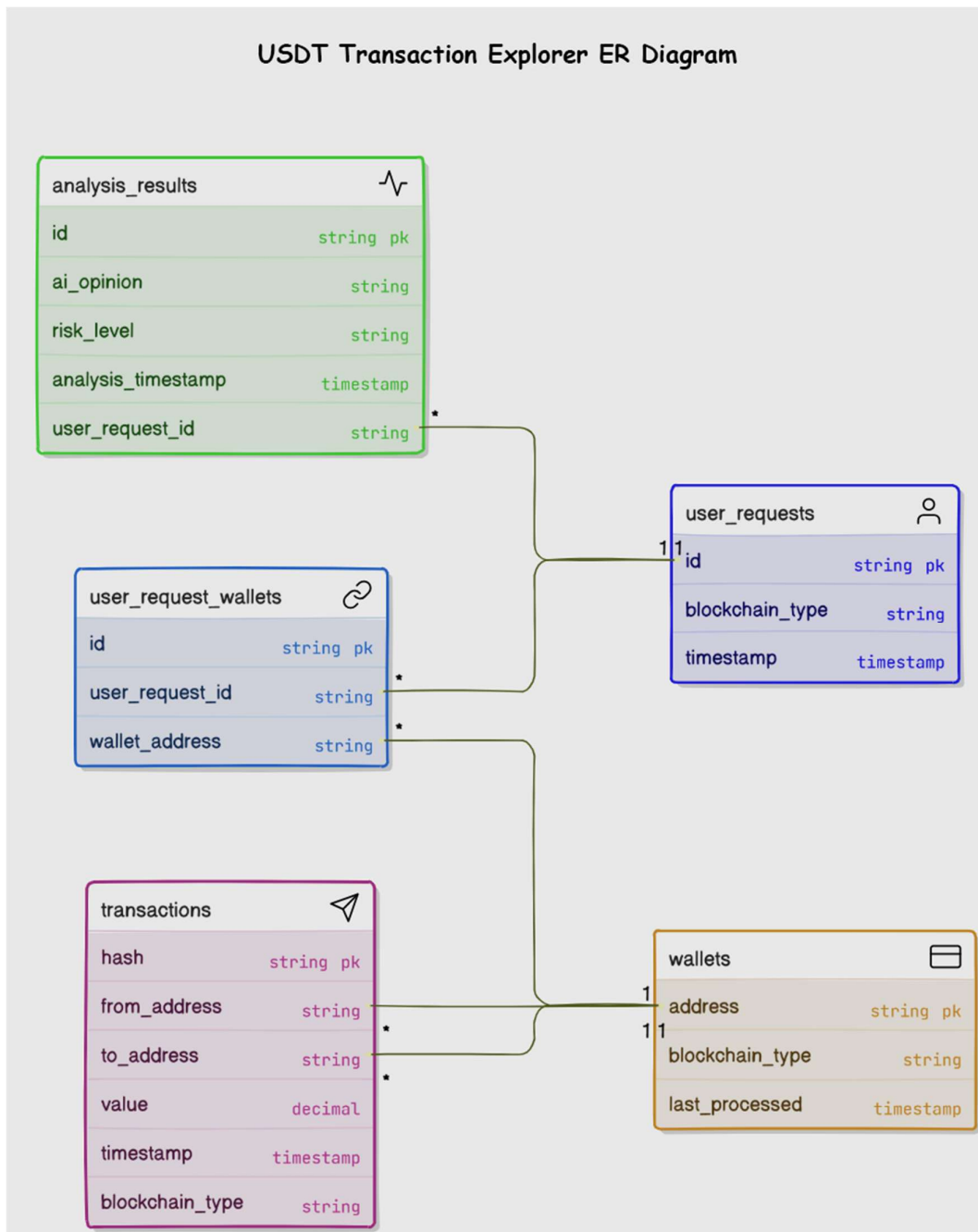
Data Stores:

- Request Progress Store - Maintains progress information for ongoing requests
- Results Store - Caches completed transaction trees and analysis results

USDT Transaction Explorer Data Flow Diagram



5.3 Entity Relationship (ER) Diagram



The ER Diagram represents the conceptual data model of the system, showing the relationships between:

1. **User Request Entity**
 - Attributes: RequestID, WalletAddress, BlockchainType, Timestamp
 - Primary key: RequestID
2. **Transaction Entity**
 - Attributes: TransactionHash, FromAddress, ToAddress, Value, Timestamp, BlockchainType

- Primary key: TransactionHash
- Foreign keys: FromAddress, ToAddress (references Wallet)
- 3. **Wallet Entity**
 - Attributes: Address, BlockchainType, LastProcessed
 - Primary key: Address
- 4. **Analysis Result Entity**
 - Attributes: AnalysisID, RequestID, AIOpinion, RiskLevel, AnalysisTimestamp
 - Primary key: AnalysisID
 - Foreign key: RequestID (references User Request)

The relationships show that:

- A User Request pertains to one specific Wallet
- A Wallet can be involved in multiple Transactions (as sender or receiver)
- A Transaction involves exactly two Wallets (sender and receiver)
- Each User Request generates one Analysis Result

5.4 Use Case Diagram

The Use Case Diagram depicts the interactions between actors and the system:

Primary Actor: User

- Use Cases:
 - Enter Wallet Address
 - View Transaction Tree
 - Analyze Wallet Risk
 - Interact with Visualization
 - Export Results

Secondary Actor: System Administrator

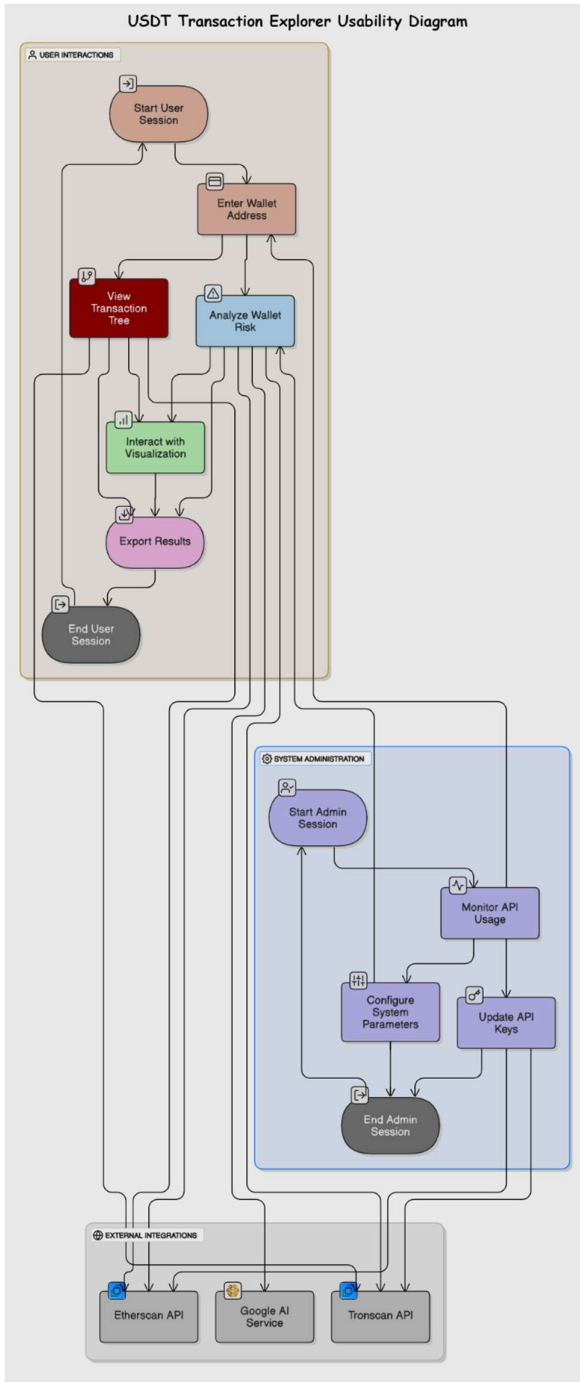
- Use Cases:
 - Monitor API Usage
 - Update API Keys
 - Configure System Parameters

External Systems as Actors:

- Etherscan API
- Tronscan API
- Google AI Service

The diagram also shows the relationships and dependencies between these use cases, highlighting how the core functionalities of wallet analysis and visualization build upon the foundation of address input and data retrieval.

USDT Transaction Explorer Usability Diagram



6. Implementation

6.1 Architecture

The USDT Transaction Explorer follows a client-server architecture with clear separation of concerns between frontend and backend components. The implementation consists of three main parts:

1. **Backend Server (app.py)**: A Flask-based RESTful API server that handles data retrieval, processing, and AI analysis
2. **Frontend Client (graph.html)**: A web interface built with HTML, CSS, and JavaScript that provides user interaction and visualization
3. **AI Module (ai.py)**: A specialized component that interfaces with Google's Generative AI to provide transaction analysis

The system utilizes asynchronous processing through Python's threading module to ensure responsiveness while handling potentially time-consuming blockchain data retrieval and processing operations.

6.2 Backend Implementation

The backend is implemented in Python using the Flask framework, providing a RESTful API interface for the frontend client. Key components include:

6.2.1 API Endpoints

The server exposes several endpoints:

- `/transaction-tree/erc20`: Initiates transaction tree generation for ERC20 addresses
- `/transaction-tree/progress`: Returns the current progress of a processing request
- `/transaction-tree/result`: Returns the completed transaction tree
- `/transaction-tree/get-ai-analysis`: Returns the AI-generated analysis for a wallet
- `/`: Serves the main HTML interface

6.2.2 Blockchain Data Retrieval

The system interfaces with blockchain explorer APIs to retrieve transaction data:

```
python
```

```
def fetch_erc20_transactions(address, request_id):  
    """Fetch ERC20 transactions for a wallet using Etherscan API or the  
    stub JSON file."""  
    global last_request_time  
    try:
```

```

if USE_STUB:
    with open(STUB_FILE_PATH, "r") as f:
        data = json.load(f)
    return data

url = f"https://api.etherscan.io/api"
params = {
    "module": "account",
    "action": "tokentx",
    "address": address,
    "apikey": ETHERSCAN_API_KEY,
    "sort": "desc", # Get most recent transactions first
}

# Handle rate limiting
while time.time() - last_request_time < 1 / API_RATE_LIMIT:
    time.sleep(0.1)

last_request_time = time.time()

response = requests.get(url, params=params)
# Process response...

```

Similar functionality exists for TRON blockchain transactions.

6.2.3 Transaction Tree Building

The core algorithm builds a transaction tree by recursively traversing related transactions:

```

python
def build_transaction_tree(address, level=7, current_level=0,
    request_id=None, processed_addresses=None, root_wallet=None,
    chain="ERC20"):
    """
    Recursively build a transaction tree, traversing both incoming and
    outgoing transactions
    up to the specified level.
    """
    try:
        # Initialize tracking sets and variables

```

```

if processed_addresses is None:
    processed_addresses = set()
if root_wallet is None:
    root_wallet = address

# Normalize current address
address = address

# Create a base node for the current address
tree = {"address": address, "transactions": []}

# Stop conditions
if current_level >= level:
    return tree

# Mark address as processed for this branch
current_branch = processed_addresses.copy()
current_branch.add(address)

# Fetch transactions
if chain == "ERC20":
    transactions = fetch_erc20_transactions(address,
request_id)
elif chain == "TRON":
    transactions = fetch_tron_transactions(address, request_id)

# Process transactions...

```

6.2.4 Asynchronous Processing

To maintain responsiveness, the system uses threading for long-running operations:

```

python
@app.route('/transaction-tree/erc20', methods=['GET'])
def transaction_tree_erc20():
    """API endpoint to generate transaction tree for ERC20."""
    try:
        wallet_address = request.args.get('address')
        if not wallet_address:

```

```

        return jsonify({"error": "Missing required parameter:
address"}), 400

        # Generate a unique request ID for tracking
        request_id = str(time.time())

        # Start the processing in a separate thread so the API remains
        responsive
        thread = Thread(target=process_request, args=(wallet_address,
request_id))
        thread.start()

        return jsonify({"request_id": request_id, "status": "Processing
started"}), 202

    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

6.2.5 AI Integration

The system integrates with Google's Generative AI for transaction analysis:

python

```

def google_ai (data):
    genai.configure(api_key=API_KEY)
    model = genai.GenerativeModel("gemini-1.5-flash")
    prompt = f'''
    Ok, so i own a business where we do frequent transaction with high profile clients using crypto wallets, i have a program that
    gives me analysis of wallet.
    This data is essential to ensure safety of my business that am dealing with a proper client.
    Talk in third person.
    Your existing responses are extremely salty, be rational and pragmatic. It can not be always money laundering, sometimes, it is actual transaction.
    I have given you a list of wallet trees who they dealt with having top 2 recent transactions from them.
    Give me analysis of this data in 5 lines as a suggestion to whether doing business with this wallet is safe or not.
    Point out the red flags. if the address starts with 0x it is from ERC20 chain and if it starts with T, it is from TRON chain
    donot suggest me to do further analysis and stuff as this is the data we got. say your opinion out loud do not tell us to do further analysis. You analyze and suggest your opinion.
    \n\n
    {data}
    ...

    response = model.generate_content(prompt)
    return response.text

```

6.3 Frontend Implementation

The frontend is implemented using HTML, CSS, and JavaScript with D3.js for visualization.

6.3.1 User Interface Design

The interface features a clean, modern design with a dark theme and gradient accents:

html

```

<style>
  :root {
    --primary: #6366f1;
    --secondary: #a855f7;
    --background: #0f172a;
    --text: #e2e8f0;
    --card: #1e293b;
  }

  body {
    background: var(--background);
    color: var(--text);
    min-height: 100vh;
    padding: 2rem;
  }

  .container {
    max-width: 1200px;
    margin: 0 auto;
  }

  h1 {
    font-size: 2.5rem;
    background: linear-gradient(to right, var(--primary), var(--secondary));
    -webkit-background-clip: text;
    background-clip: text;
    color: transparent;
    margin-bottom: 2rem;
  }

  /* Additional styles... */
</style>

```

6.3.2 API Communication

The frontend communicates with the backend API using fetch requests:

```
javascript
```

```

document.getElementById('addressForm').addEventListener('submit',
async function (e) {
    e.preventDefault();
    const address = document.getElementById('address').value;
    const progressDiv = document.getElementById('progress');
    const completionDiv = document.getElementById('completion');
    const walletAnalysis = document.getElementById('walletAnalysis');

    progressDiv.style.display = 'block';
    completionDiv.style.display = 'none';
    walletAnalysis.style.display = 'none';

    // Determine blockchain type based on address prefix
    const blockchainType = address.startsWith('T') ? 'tron' : 'erc20';
    const endpoint = `/transaction-tree/erc20`;

    const res = await
fetch(`${endpoint}?address=${encodeURIComponent(address)}`);
    const data = await res.json();

    if (data.request_id) {
        const requestId = data.request_id;
        const progressInterval = setInterval(async function () {
            // Poll for progress updates...
        }, 1000);
    }
});

```

6.3.3 D3.js Visualization

The system uses D3.js to create interactive transaction tree visualizations:

```

javascript
function drawTree(data, blockchainType) {
    const root = {
        name: data.address,
        value: 1,
        children: transformData(data || []),
        blockchain: blockchainType
    };
}

```

```

};

const margin = { top: 40, right: 120, bottom: 40, left: 120 };
const width = window.innerWidth - margin.left - margin.right - 100;
const height = 800 - margin.top - margin.bottom;

const svg = d3.select("#graph svg");
svg.selectAll("*").remove();

// Create gradient definitions
svg.append("defs")
  .append("linearGradient")
  .attr("id", "gradient")
  .attr("x1", "0%").attr("y1", "0%")
  .attr("x2", "100%").attr("y2", "0%")
  .selectAll("stop")
  .data([
    { offset: "0%", color: "#6366f1" },
    { offset: "100%", color: "#a855f7" }
  ])
  .enter().append("stop")
  .attr("offset", d => d.offset)
  .attr("stop-color", d => d.color);

// Create zoom container
const g = svg.append("g")
  .attr("class", "zoom-container")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// Create tree layout
const tree = d3.tree().size([height, width]);
const rootNode = d3.hierarchy(root);

// Calculate node sizes
const maxValue = d3.max(rootNode.descendants().slice(1), d => {
  const value = parseFloat(d.data.value) || 0;
  return value * 1e18;
});

```

```

});

const radiusScale = d3.scaleLinear()
  .domain([0, maxValue])
  .range([5, 25]);

// Apply tree layout
tree(rootNode);

// Render links
const links = g.selectAll(".link")
  .data(rootNode.links())
  .enter()
  .append("path")
  .attr("class", "link")
  .attr("d", d3.linkHorizontal()
    .x(d => d.y)
    .y(d => d.x));

// Additional visualization code...
}

```

6.3.4 Interactive Features

The visualization includes several interactive features:

```

javascript
// Add zoom behavior
const zoom = d3.zoom()
  .scaleExtent([0.1, 4])
  .on("zoom", (event) => {
    currentZoom = event.transform;
    g.attr("transform",
      `translate(${event.transform.x
margin.left},${event.transform.y + margin.top})
      scale(${event.transform.k})`
    );
  });

```

```

svg.call(zoom);

// Add drag behavior
const drag = d3.drag()
  .on("start", dragstarted)
  .on("drag", dragged)
  .on("end", dragended);

nodes.call(drag);

function dragstarted(event) {
  d3.select(this).raise().classed("active", true);
}

function dragged(event, d) {
  const dx = event.dx / currentZoom.k;
  const dy = event.dy / currentZoom.k;

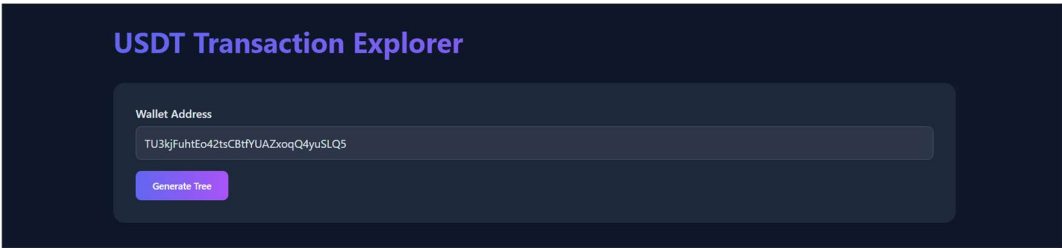
  d.x += dy;
  d.y += dx;

  d3.select(this)
    .attr("transform", `translate(${d.y},${d.x})`);

  links
    .filter(l => l.source === d || l.target === d)
    .attr("d", d3.linkHorizontal()
      .x(d => d.y)
      .y(d => d.x));
}

```

6.4 Interface Screenshots



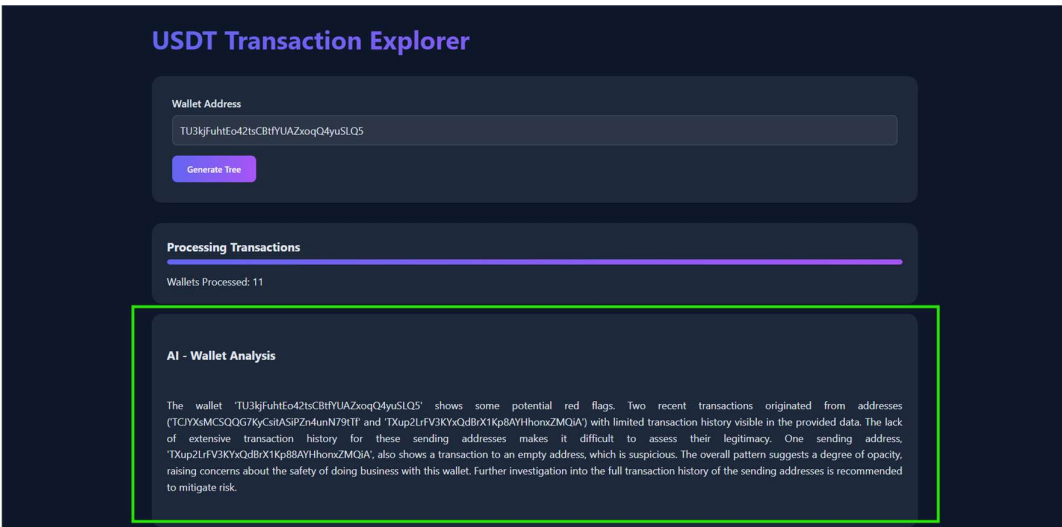
Snapshot: Main Input Screen

The main interface allows users to enter a wallet address and initiate the transaction tree generation process.



Snapshot: Progress Tracking

During processing, users see a progress bar and statistics on the number of wallets processed.



Snapshot: AI Analysis Display

The AI analysis section shows the Gemini model's assessment of the wallet's risk profile and transaction patterns.

6. Conclusion, Limitations and Future Scope

7.1 Conclusion

The USDT Transaction Explorer represents a significant advancement in cryptocurrency transaction analysis by combining blockchain data retrieval, interactive visualization, and AI-powered insights. The project successfully addresses the key challenges of transaction transparency, cross-chain analysis, and risk assessment in cryptocurrency operations.

The implementation demonstrates the viability of an integrated approach to blockchain analytics, making complex transaction data accessible and interpretable for users across different technical backgrounds. By visualizing transaction networks and providing automated risk analysis, the system enhances decision-making capabilities for businesses and individuals engaging with cryptocurrency wallets.

The combination of Flask-based backend services with D3.js visualizations proved effective in handling the unique challenges of blockchain data processing and presentation. The integration of Google's Generative AI further enhanced the system's capabilities by providing contextual analysis of transaction patterns, transforming raw data into actionable insights.

7.2 Limitations

Despite its capabilities, the current implementation has several limitations:

1. **API Dependency:** The system relies heavily on third-party blockchain explorer APIs (Etherscan and Tronscan), making it vulnerable to rate limits, service disruptions, or API changes.
2. **Scalability Constraints:** The recursive transaction tree building algorithm may face performance issues with wallets that have extensive transaction histories or complex relationship networks.
3. **Limited Blockchain Support:** Currently, the system supports only ERC20 (Ethereum) and TRON blockchains, while USDT operates on additional networks including Solana, Algorand, and others.
4. **Visualization Performance:** The D3.js visualization may experience performance degradation when rendering very large transaction networks with hundreds of nodes.
5. **AI Analysis Depth:** The current AI analysis is limited to a brief assessment based on available transaction data, without incorporating additional context or historical patterns that might enhance accuracy.
6. **Privacy Considerations:** While all blockchain data used is publicly available, the visualization and analysis of transaction networks may raise privacy concerns for users who expect pseudonymity in their cryptocurrency operations.
7. **Limited Historical Data:** The system currently focuses on recent transactions rather than providing comprehensive historical analysis, potentially missing important long-term patterns.

7.3 Future Scope

Several improvements could address the limitations and expand the system's capabilities:

1. **Extended Blockchain Support:** Integrating additional blockchain networks such as Solana, Binance Smart Chain, and Algorand to provide truly comprehensive USDT transaction analysis.
2. **Direct Node Connection:** Implementing direct blockchain node connections instead of relying solely on third-party APIs to improve reliability and remove rate limitations.
3. **Advanced Analytics Features:** Incorporating statistical analysis tools, anomaly detection algorithms, and pattern recognition to enhance the system's analytical capabilities.
4. **Enhanced Visualization Options:** Adding alternative visualization modes (e.g., force-directed graphs, heatmaps, timeline views) to provide different perspectives on transaction data.
5. **Machine Learning Integration:** Developing specialized machine learning models trained on cryptocurrency transaction data to improve risk assessment accuracy and identify sophisticated patterns.
6. **Batch Processing Capabilities:** Adding support for analyzing multiple wallets simultaneously to identify relationships and patterns across a broader network.
7. **Export and Reporting Features:** Implementing comprehensive reporting functions that allow users to export analysis results in various formats for compliance or research purposes.
8. **Real-time Monitoring:** Evolving the system from on-demand analysis to continuous monitoring of specified wallets with alert mechanisms for suspicious activities.
9. **Collaborative Analysis:** Adding features that enable team-based investigation of transaction networks with annotation and sharing capabilities.
10. **Mobile Application:** Developing a companion mobile application for on-the-go transaction monitoring and alerts.

By addressing these future development areas, the USDT Transaction Explorer could evolve into a comprehensive cryptocurrency intelligence platform serving diverse stakeholders in the digital asset ecosystem.

References

Akcora, C. G., Dey, A. K., Gel, Y. R., & Kantarcioglu, M. (2019). Forecasting Bitcoin price with graph chainlets. In Pacific-Asia Conference on Knowledge Discovery and Data Mining (pp. 765-776).

Chen, W., Zheng, Z., Cui, J., Ngai, E., Zheng, P., & Zhou, Y. (2022). Detecting Ponzi schemes on Ethereum: Towards healthier blockchain technology. In Proceedings of the 2022 World Wide Web Conference (pp. 1-11).

Kentaroh Toyoda, Xiao Wang, Mingzhe Li, Bo Gao, Yuan Wang, and Qingsong Wei (2024). Blockchain Data Analysis in the Era of Large-Language Models